

PRÁCTICA: ANÁLISIS DE DATOS Y ETL

Data Warehouse, Limpieza y Minería de Datos

Administración de Bases de Datos

Estudiante:

Miguel Angel Hernandez Cervantes

Grupo:

IDGS – 93

Cuatrimestre:

9°

Profesor:

Héctor Velazquez Estrada

PARTE I — ANÁLISIS TEÓRICO (40%)

Actividad 1. Tipos y Fuentes de Datos

1.1 Clasificación de fuentes de datos

Fuente	Descripción	Tipo	Clasificación
ventas.csv	Ventas diarias	CSV (tabular)	Estructurada
clientes.json	Info de clientes	JSON (anidado)	Semi-estructurada
comentarios.txt	Opiniones de clientes	Texto plano	No estructurada
productos.xlsx	Catálogo de productos	Excel (hoja)	Estructurada

1.2 Justificación de la clasificación

ventas.csv — Estructurada: filas y columnas con esquema fijo, tipos de dato predefinidos y relaciones implícitas entre campos. Directamente consumible por motores SQL y pandas.

clientes.json — Semi-estructurada: posee metadatos (claves) que describen el contenido, pero puede incluir campos anidados, arrays y valores nulos variables sin un esquema rígido obligatorio.

comentarios.txt — No estructurada: texto libre sin delimitadores, jerarquía ni esquema. Requiere NLP (tokenización, análisis de sentimientos) para extraer información cuantificable.

productos.xlsx — Estructurada: tablas relacionales con encabezados explícitos, celdas tipadas y fórmulas opcionales. Compatible con todo motor tabular.

1.3 Ventajas y desventajas por tipo

Tipo	Ventajas	Desventajas
Estructurada	Consultas SQL directas, fácil integración con DWH, alta eficiencia en almacenamiento y compresión	Esquema rígido dificulta la evolución del modelo; no apto para datos libres o multimedia
Semi-estructurada	Flexible, autodescriptiva, soporta jerarquías y evolución de esquema sin migraciones	Mayor complejidad en consultas (JSONPath, XPath), joins costosos, parseo adicional
No estructurada	Captura contexto humano, opiniones y emociones; fuente clave para ML y análisis de sentimientos	Procesamiento intensivo (NLP), difícil de validar, alta varianza semántica y ruido

Actividad 2. Modelado de Data Warehouse

2.1 Modelo Estrella (Star Schema)

El modelo estrella centra el esquema en una tabla de hechos (FACT_VENTAS) conectada directamente a tablas de dimensiones desnormalizadas. Minimiza los JOINS y favorece el rendimiento en consultas OLAP.

Tabla de hechos: FACT_VENTAS

- venta_id (PK), fecha_id (FK), cliente_id (FK), producto_id (FK), cantidad, precio_unitario, descuento, total

Dimensiones (desnormalizadas):

- DIM_FECHA: fecha_id, fecha, dia, mes, trimestre, anio, dia_semana

- DIM_CLIENTE: cliente_id, nombre, email, ciudad, edad, genero
- DIM_PRODUCTO: producto_id, nombre, categoria, precio_base, proveedor

2.2 Modelo Copo de Nieve (Snowflake Schema)

El modelo copo de nieve normaliza las dimensiones en subdimensiones, reduciendo redundancia a costa de más JOINS. Adecuado cuando el almacenamiento es crítico y los datos de dimensión son muy grandes.

- DIM_PRODUCTO se descompone en: DIM_PRODUCTO (producto_id, nombre, precio, categoria_id) + DIM_CATEGORIA (categoria_id, nombre_cat, descripcion)
- DIM_CLIENTE se descompone en: DIM_CLIENTE (cliente_id, nombre, email, ciudad_id) + DIM_CIUADAD (ciudad_id, ciudad, estado, pais)
- DIM_FECHA se mantiene sin subdivisiones adicionales en este caso

2.3 Comparación entre modelos

Criterio	Estrella	Copo de nieve
Normalización	Dimensiones desnormalizadas (1NF–2NF)	Dimensiones en 3NF, subdimensiones
Rendimiento queries	Alto: menos JOINS	Menor: más JOINS necesarios
Uso de espacio	Mayor por redundancia	Menor, sin redundancia
Mantenimiento	Simple, una tabla por dimensión	Complejo, múltiples tablas
Uso recomendado	OLAP, BI, dashboards	DWH empresarial con dimensiones grandes

Actividad 3. Técnicas de Limpieza de Datos

3.1 Técnicas básicas

Eliminación de duplicados

Registros idénticos en todas o en columnas clave generan sobreestimación en métricas. Se eliminan comparando hashes de fila o subconjuntos de columnas clave.

Implementación pandas: df.drop_duplicates() | PySpark: df.dropDuplicates(['id','fecha'])

Tratamiento de nulos

Estrategia según tipo de columna: numérica → mediana (robusta ante outliers); categórica → moda o 'Desconocido'; timestamp → interpolación o eliminación de fila.

Corrección de formatos

Unificar representaciones: fechas a ISO 8601, monedas sin símbolo, texto en minúsculas/strip. Crítico antes de joins entre fuentes heterogéneas.

Estandarización

Normalizar unidades de medida, escalas numéricas y vocabulario controlado (e.g., 'Méx', 'México DF' → 'Ciudad de México'). Reduce cardinalidad en dimensiones.

3.2 Técnicas avanzadas

Detección de outliers

IQR: valores fuera de $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ son candidatos a revisión. Z-Score: $|z| > 3$ indica anomalía estadística. DBSCAN para detección multivariada.

Imputación estadística

KNN Imputer: imputa con la media de los k vecinos más cercanos. IterativeImputer (MICE): modela cada columna como función de las demás. Aplicable cuando la ausencia es MAR (Missing At Random).

Validación cruzada de limpieza

Entrenar modelo base antes y después de la imputación; comparar métricas (R^2 , MAE). Un deterioro indica imputación ruidosa; una mejora confirma la calidad de la estrategia.

Normalización

Min-Max Scaling: escala a $[0, 1]$, sensible a outliers. StandardScaler: media 0, desviación estándar 1, recomendado para algoritmos basados en distancia (KNN, SVM, PCA).

Actividad 4. Minería de Datos

4.1 Definición de minería de datos (en mis propias palabras)

La minería de datos es el proceso de descubrir patrones, correlaciones y anomalías en conjuntos de datos grandes aplicando métodos estadísticos y de machine learning. A diferencia de una consulta SQL, no parte de una hipótesis predefinida; el objetivo es dejar que los datos revelen estructuras desconocidas que generen valor de negocio.

4.2 Objetivos (en mis propias palabras)

- Convertir datos históricos en modelos predictivos accionables.
- Segmentar clientes o productos para personalizar estrategias.
- Detectar comportamientos atípicos (fraude, fallas, churn).
- Reducir dimensionalidad y extraer las variables más influyentes.
- Automatizar decisiones mediante modelos entrenados y validados.

4.3 Aplicaciones empresariales

Industria	Aplicación	Técnica principal
Retail	Market basket analysis, recomendaciones	Apriori, Reglas de asociación
Banca	Detección de fraude en tiempo real	Isolation Forest, Autoencoders
Salud	Predicción de diagnóstico clínico	Random Forest, XGBoost
Manufactura	Mantenimiento predictivo (PdM)	LSTM, redes neuronales temporales
e-Commerce	Predicción de churn de clientes	Regresión logística, SVM

4.4 Proceso KDD (Knowledge Discovery in Databases)

El proceso KDD es el ciclo formal de extracción de conocimiento desde datos en bruto:

1. SELECCIÓN — Identificar fuentes de datos relevantes para el problema de negocio.
2. PREPROCESAMIENTO — Limpieza, integración y tratamiento de nulos/outliers.
3. TRANSFORMACIÓN — Reducción dimensional, encoding, normalización, feature engineering.
4. MINERÍA DE DATOS — Aplicación de algoritmos (clasificación, clustering, regresión, asociación).

- 5. INTERPRETACIÓN / EVALUACIÓN — Validar el modelo, medir métricas (AUC, F1, RMSE) y traducir resultados a lenguaje de negocio.
- 6. CONOCIMIENTO — Integrar el modelo en procesos de toma de decisión; monitoreo continuo (data drift).

PARTE II — DESARROLLO PRÁCTICO (40%)

Actividad 5. Implementación ETL con Python / PySpark

El código completo se encuentra en los notebooks adjuntos al ZIP:

- notebooks/etl_pipeline.py — ETL completo con pandas + scikit-learn
- notebooks/etl_pyspark.py — ETL equivalente con PySpark + MLlib
- data/ — CSVs y Parquet generados por el pipeline

5.1 Paso 1 — Extracción

Se generan cuatro datasets sintéticos realistas con la librería Faker (locale es_MX) y numpy para simular nulos, duplicados y outliers. El DataFrame de ventas contiene 500 registros base más 20 duplicados intencionales, lo que lo hace representativo para demostrar limpieza.

Archivo	Registros	Campos clave
ventas.csv	520 (con 20 dupl.)	venta_id, fecha, cliente_id, producto, categoria, cantidad, precio_unitario, descuento
productos.xlsx	40	producto_id, nombre, categoria, precio, inventario, proveedor, fecha_alta
clientes.json	100	cliente_id, nombre, email, ciudad, edad, genero, registro
comentarios.txt	50 líneas	Texto libre con etiqueta de sentimiento (positivo/negativo/neutro)

5.2 Paso 2 — Transformación

- Eliminación de duplicados: `df.drop_duplicates()` → 20 registros removidos.
- Imputación de nulos numéricos: mediana de columna (robusta ante distribuciones sesgadas).
- Imputación categórica: 'Desconocido' para nombre, moda para género.
- Casting de tipos: fecha → `datetime64`, cantidad → `int`, precio → `float64`.
- Feature derivada: $\text{total} = \text{cantidad} \times \text{precio_unitario} \times (1 - \text{descuento})$.
- PySpark equivalente: `approxQuantile()` para medianas en distribución, `fillna()` y `withColumn()` para transformaciones.

5.3 Paso 3 — Carga

Los DataFrames limpios se persisten en formato Apache Parquet. Justificación: almacenamiento columnar con compresión Snappy (~70% menos espacio vs CSV), lectura eficiente por predicado pushdown y esquema embebido (evita re-inferencia).

PARTE III — MINERÍA DE DATOS (20%)

Actividad 6. Análisis Exploratorio

Los resultados son generados dinámicamente por el pipeline a partir del dataset sintético. Resultados de ejecución:

Pregunta analítica	Resultado	Implicación de negocio
¿Cuál es el producto más caro?	Avena — \$7,658.05	Revisar estrategia de pricing en Alimentos
¿Cuál tiene mayor inventario?	Tapete — 486 unidades	Posible sobrestock, evaluar rotación
¿Mayor valor económico en inventario?	Tapete — \$3,001,258.98	Prioridad en control de inventario físico
Categoría con más ventas totales	Alimentos — \$2,487,095	Potencial para campañas de cross-selling

Actividad 7. Predicción Simple — Regresión de Valor de Inventario

Objetivo: predecir valor_inventario (precio × inventario) a partir de precio, inventario y categoría, simulando un caso real de planificación financiera de almacén.

7.1 Feature engineering

- LabelEncoder para categoría (StringIndexer en PySpark).
- Features: precio (float), inventario (int), cat_enc (int codificado).
- Target: valor_inventario = precio × inventario.

7.2 Resultados de modelos

Modelo	MAE	R ² Score
LinearRegression	\$291,914.92	0.6734
GradientBoosting	\$222,258.58	0.8143

El modelo GradientBoosting supera a la regresión lineal con $R^2 = 0.81$, lo que indica que el 81% de la varianza del valor de inventario es explicada por las tres variables de entrada. El MAE de \$222K sobre un target con rango de hasta \$3M es aceptable para un dataset de 40 productos.

7.3 Análisis de resultados

- La relación precio × inventario no es perfectamente lineal: existe interacción entre categoría y precio que los árboles capturan mejor.
- Para producción se recomendaría: más registros históricos, incorporar estacionalidad y proveedor como features, y usar cross-validation con k=5.
- Versión PySpark MLlib (etl_pyspark.py): mismo pipeline con LinearRegression y GBRegressor de org.apache.spark.ml.regression.

Actividad 8. Evidencia de la Ejecución

```
Archivo Editar Selección Ver Ir ... AdminBaseDatos [WSL: Ubuntu]
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
bash - practica_etl

(venv) migue@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
✓ Datos generados correctamente
ventas.csv → 520 registros (con duplicados)
productos.xlsx → 40 registros
clientes.json → 100 registros
comentarios.txt → 50 comentarios

=====
PASO 2 – TRANSFORMACIÓN (pandas)
=====

[ventas] shape antes: (520, 8)
venta_id      int64
fecha         object
cliente_id    int64
producto      object
categoria     object
cantidad      float64
precio_unitario float64
descuento     float64
dtype: object

Duplicados eliminados: 20
cantidad: 238 nulos imputados con mediana=11.00
precio_unitario: 21 nulos imputados con mediana=2485.67
productos.precio: 3 nulos imputados con mediana=3636.45
productos.inventario: 2 nulos imputados con mediana=237.50

[ventas] shape después: (500, 9)
  venta_id  fecha  cliente_id  producto  categoria  cantidad  precio_unitario  descuento  total
0         1  2026-04-30         50   Reloj      Hogar         11.0         3102.71         0.23  26279.9537
1         2  2026-03-09         62  Webcam  Electrónica      12.0         1778.66         0.03  20703.6024
2         3  2025-09-14         14  Playera      Ropa         11.0          404.31         0.04   4269.5136

✓ Datos limpios guardados en formato Parquet (compatible Spark)

=====
PARTE III – ANÁLISIS EXPLORATORIO
=====

¿Cuál es el producto más caro?
```

```
Archivo Editar Selección Ver Ir ... AdminBaseDatos [WSL: Ubuntu]
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
bash - practica_etl

(venv) migue@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py

=====
PARTE III – ANÁLISIS EXPLORATORIO
=====

¿Cuál es el producto más caro?
→ Avena (Deportes) – $7658.05

¿Cuál tiene mayor inventario?
→ Tapete (Deportes) – 486 unidades

¿Cuál representa mayor valor económico en inventario?
→ Tapete (Deportes) – $3,001,258.98

--- Estadísticas generales de productos ---
      precio  inventario  valor_inventario
count    40.00         40.00             40.00
mean    3694.88        235.30        903576.07
std     2083.26        150.63        885885.39
min     238.84          2.00         5192.98
25%    2285.05        103.25        172812.30
50%    3636.45        237.50        574863.78
75%    5239.83        345.00       1309583.79
max     7658.05        486.00       3001258.98

--- Ventas totales por categoría (Top 5) ---
categoria
Alimentos    2.487095e+06
Ropa         2.472554e+06
Deportes     2.229875e+06
Hogar        2.063723e+06
Electrónica  2.031299e+06

=====
ACTIVIDAD 7 – PREDICCIÓN SIMPLE (scikit-learn)
=====

LinearRegression → MAE: 291,914.92 | R²: 0.6734
GradientBoosting → MAE: 222,258.58 | R²: 0.8143

--- DataFrame de predicciones (muestra) ---
```

```
Archivo Editar Selección Ver Ir ... AdminBaseDatos [WSL: Ubuntu]
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
bash - practica_etl

(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
=====
ACTIVIDAD 7 - PREDICCIÓN SIMPLE (scikit-learn)
=====

LinearRegression → MAE: 291,914.92 | R²: 0.6734
GradientBoosting → MAE: 222,258.58 | R²: 0.8143

--- DataFrame de predicciones (muestra) ---
   precio inventario categoria valor_real pred_lineal pred_gradboost error_lineal error_gradboost
0  4094.45      237.5  Electrónica  972431.875  1028580.05      792860.09      56148.18      179571.79
1  5805.61      407.0  Electrónica  2362883.270  2176163.58      1998417.35      186719.69      364465.92
2   951.32      454.0        Hogar    431899.280  1220847.62      967252.18      788948.34      535352.90
3  5498.97      108.0  Electrónica  593888.760   795327.35      379641.84      201438.59      214246.92
4  2827.75       54.0        Ropa   152698.500   -92841.90      430107.08      245540.40      277408.58
5  1988.43      408.0  Alimentos  811279.440  1292796.83      645127.71      481517.39      166151.73
6  2565.41       50.0        Ropa   128270.500  -172593.38      139906.83      300863.88      11636.33
7  1684.36      330.0  Deportes  555838.800   870376.44      899878.36      314537.64      344039.56
8  3636.45      169.0  Alimentos  614560.050   638282.65      710732.34      23722.60      96172.29
9  2383.92      44.0  Deportes  104892.480  -214820.01      71352.70      319712.49      33539.78

✓ Pipeline completo ejecutado
26/06/02 23:30:21 WARN Utils: Your hostname, Miguel resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/06/02 23:30:21 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/06/02 23:30:22 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
✓ Spark inicializado: 3.5.1

=====
PASO 1 - EXTRACCIÓN
=====

[ventas] schema:
root
 |-- venta_id: integer (nullable = true)
 |-- fecha: date (nullable = true)
 |-- cliente_id: integer (nullable = true)
 |-- producto: string (nullable = true)
 |-- categoria: string (nullable = true)
 |-- cantidad: double (nullable = true)
```

```
Archivo Editar Selección Ver Ir ... AdminBaseDatos [WSL: Ubuntu]
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
bash - practica_etl

(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
=====
PASO 1 - EXTRACCIÓN
=====

[ventas] schema:
root
 |-- venta_id: integer (nullable = true)
 |-- fecha: date (nullable = true)
 |-- cliente_id: integer (nullable = true)
 |-- producto: string (nullable = true)
 |-- categoria: string (nullable = true)
 |-- cantidad: double (nullable = true)
 |-- precio_unitario: double (nullable = true)
 |-- descuento: double (nullable = true)

Registros: 520 | Columnas: 8

[productos] registros: 40
[clientes] registros: 100

=====
PASO 2 - TRANSFORMACIÓN (PySpark)
=====

Duplicados eliminados: 20
cantidad imputada con mediana=11.0
precio_unitario imputada con mediana=2452.53

--- DataFrame ventas (limpio) ---
+-----+-----+-----+-----+-----+-----+-----+-----+
|venta_id|fecha      |cliente_id|producto|categoria|cantidad|precio_unitario|descuento|total      |
+-----+-----+-----+-----+-----+-----+-----+-----+
|8       |2025-06-20|74       |Laptop  |Electrónica|16      |168.11         |0.03     |2609.0672  |
|9       |2025-08-07|38       |Arroz    |Alimentos  |19      |1892.08        |0.23     |27681.1304 |
|30      |2026-03-13|29       |Tapete   |Hogar      |15      |1735.84        |0.1       |23433.84    |
|34      |2026-04-20|33       |Pelota   |Deportes   |14      |1638.62        |0.26     |16976.1032 |
|38      |2025-06-27|99       |Reloj    |Hogar      |18      |2933.52        |0.09     |48051.0576 |
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```



```
(venv) miguel@Miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
root
|-- venta_id: integer (nullable = true)
|-- fecha: date (nullable = true)
|-- cliente_id: integer (nullable = true)
|-- producto: string (nullable = true)
|-- categoria: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unitario: double (nullable = false)
|-- descuento: double (nullable = true)
|-- total: double (nullable = true)

=====
PASO 3 - CARGA
=====
✓ ventas limpias guardadas en Parquet (PySpark)

=====
ANÁLISIS EXPLORATORIO - PySpark
=====

¿Producto más caro?
+-----+-----+
|nombre|categoria|precio |
+-----+-----+
|Avena |Deportes |7658.05|
+-----+-----+
only showing top 1 row

¿Producto con mayor inventario?
+-----+-----+
|nombre|categoria|inventario|
+-----+-----+
|Tapete|Deportes |486.0    |
+-----+-----+
only showing top 1 row

¿Mayor valor económico en inventario?
+-----+-----+
|nombre|categoria|precio |inventario|valor_inventario|
+-----+-----+
```

```
Archivo Editor Selección Ver Ira ... AdminBaseDatos [WSL: Ubuntu]
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
(venv) miguel@Miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
¿Mayor valor económico en inventario?
+-----+-----+-----+-----+
|nombre|categoria|precio |inventario|valor_inventario|
+-----+-----+-----+-----+
|Tapete|Deportes |6175.43|486.0   |3001258.98      |
+-----+-----+-----+-----+
only showing top 1 row

Ventas totales por categoria:
+-----+-----+-----+
| categoria|total_ventas|num_ventas|
+-----+-----+-----+
| Alimentos|2486063.77  |99|
| Ropa      |2471456.98  |110|
| Deportes  |2228589.38  |107|
| Hogar     |2061819.24  |95|
| Electrónica|2029982.58  |89|
+-----+-----+-----+

=====
ACTIVIDAD 7 – REGRESIÓN (PySpark MLlib)
=====

LinearRegression:
MAE = 165,800.65 | R² = 0.9358

GBRegressor:
MAE = 240,300.20 | R² = 0.7598

--- DataFrame de predicciones (GBT) ---
+-----+-----+-----+-----+-----+-----+
|nombre |categoria|precio |inventario|valor_real|prediccion|
+-----+-----+-----+-----+-----+-----+
|Avena  |Electrónica|6300.75|237.5   |1496428.13|782664.51|
|Tenis  |Deportes   |2383.92|44.0    |104892.48 |170512.24|
|Guantes|Ropa        |4512.61|202.0   |911547.22 |778627.62|
|Cereal |Hogar       |5211.94|15.0    |78179.1   |98056.01 |
|Auriculares|Hogar    |3636.45|151.0   |549103.95 |593434.88|
|Monitor|Electrónica|4094.45|237.5   |972431.88 |798739.42|
```

```
Archivo  Editar  Selección  Ver  Ira  ...  AdminBaseDatos [WSL: Ubuntu]  bash - practica_etl

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
| categoria|total_ventas|num_ventas|
+-----+-----+-----+
| Alimentos| 2486063.77|    99|
| Ropa      | 2471456.98|   110|
| Deportes  | 2228589.38|   107|
| Hogar     | 2061819.24|    95|
| Electrónica| 2029982.58|    89|
+-----+-----+-----+

=====
ACTIVIDAD 7 - REGRESIÓN (PySpark MLlib)
=====

LinearRegression:
MAE = 165,800.65 | R² = 0.9358

GBRegressor:
MAE = 240,300.20 | R² = 0.7598

--- DataFrame de predicciones (GBT) ---
+-----+-----+-----+-----+-----+-----+
| nombre | categoria | precio | inventario | valor_real | predicción |
+-----+-----+-----+-----+-----+-----+
| Avena  | Electrónica| 6300.75| 237.5     | 1496428.13| 782664.51 |
| Tenis  | Deportes  | 2383.92| 44.0      | 104892.48 | 170512.24 |
| Guantes| Ropa      | 4512.61| 202.0     | 911547.22 | 778627.62 |
| Cereal | Hogar     | 5211.04| 15.0      | 78179.1   | 98056.01 |
| Auriculares| Hogar | 3636.45| 151.0     | 549103.95 | 593434.88 |
| Monitor| Electrónica| 4094.45| 237.5     | 972431.88 | 798739.42 |
| Arroz  | Electrónica| 6197.88| 484.0     | 2999773.92| 3031709.43|
| Playera| Alimentos | 3510.14| 113.0     | 396645.82 | 612463.01 |
| Cojín  | Deportes  | 623.36 | 129.0     | 80413.44  | 201774.82 |
| Yogurt | Deportes  | 351.2  | 290.0     | 101848.0  | 160097.85 |
+-----+-----+-----+-----+-----+-----+

only showing top 10 rows

✓ PySpark ETL + MLlib completado
(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$
```

```
Archivo  Editar  Selección  Ver  Ira  ...  AdminBaseDatos [WSL: Ubuntu]  bash - practica_etl

etl_pipeline.py  etl_pyspark.py

practica_etl > notebooks > etl_pipeline.py > ...
1  """
2  Práctica: Análisis de Datos - ETL Pipeline
3  Parte II y III: Implementación ETL + Minería de Datos
4  Entorno: pandas, PySpark, scikit-learn
5  """
6
7  #
8  # PASO 1 - EXTRACCIÓN: Generar datasets sintéticos
9  #
10 import pandas as pd
11 import numpy as np
12 from faker import Faker
13 import random
14 import warnings
15 from pathlib import Path
16 warnings.filterwarnings("ignore")
17
18 # Rutas relativas al script - funciona desde cualquier directorio
19 BASE_DIR = Path(__file__).resolve().parent.parent
20 DATA_DIR = BASE_DIR / "data"
21 DATA_DIR.mkdir(parents=True, exist_ok=True)
22
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
| Auriculares|Hogar| 3636.45|151.0| 549103.95 |593434.88 |
| Monitor    |Electrónica|4094.45|237.5| 972431.88 |798739.42 |
| Arroz      |Electrónica|6197.88|484.0| 2999773.92|3031709.43|
| Playera    |Alimentos |3510.14|113.0| 396645.82 |612463.01 |
| Cojín      |Deportes  |623.36 |129.0| 80413.44  |201774.82 |
| Yogurt     |Deportes  |351.2  |290.0| 101848.0  |160097.85 |
+-----+-----+-----+-----+-----+
only showing top 10 rows

✓ PySpark ETL + MLlib completado
(venv) miguel@miguel:~/abd/AdminBaseDatos/practica_etl$
```

Archivo Editar Selección Ver Ira ... AdminBasedeDatos [WSL: Ubuntu]

etl_pipeline.py etl_pyspark.py X

practica_etl > notebooks > etl_pyspark.py > ...

```
1 """
2 Práctica: Análisis de Datos – ETL con PySpark
3 Actividad 5 (variante PySpark) + Actividad 7 con MLlib
4 """
5
6 from pyspark.sql import SparkSession
7 from pyspark.sql import functions as F
8 from pyspark.sql.types import DoubleType, IntegerType
9 from pyspark.ml.feature import VectorAssembler, StringIndexer
10 from pyspark.ml.regression import LinearRegression, GBRegressor
11 from pyspark.ml import Pipeline
12 from pyspark.ml.evaluation import RegressionEvaluator
13 import warnings
14 from pathlib import Path
15 warnings.filterwarnings("ignore")
16
17 # Rutas relativas al script
18 BASE_DIR = Path(__file__).resolve().parent.parent
19 DATA_DIR = BASE_DIR / "data"
20 DATA_DIR.mkdir(parents=True, exist_ok=True)
21
22 #
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

bash - practica_etl

```
(venv) miguel@miguel:~/abd/AdminBasedeDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspark.py
|Auniculares|Hogar      |3636.45|151.0 |549103.95 |593434.88 |
|Monitor    |Electrónica|4094.45|237.5 |972431.88 |798739.42 |
|Arroz      |Electrónica|6197.88|484.0 |2999773.92|3031709.43|
|Playera    |Alimentos  |3510.14|113.0 |396645.82 |612463.01 |
|Cojin      |Deportes   |623.36 |129.0 |80413.44  |201774.82 |
|Yogurt     |Deportes   |351.2  |290.0 |101848.0  |160097.85 |
+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

✓ PySpark ETL + MLlib completado
(venv) miguel@miguel:~/abd/AdminBasedeDatos/practica_etl$
```

WSL: Ubuntu main 0 0 0 0 0 0 MonkeyFlip (Hace 7 minutos) Lin. 5, col. 1 Espacios: 4 UTF-8 LF {} Python venv (3.12.3) Prettier

Archivo Editar Selección Ver Ira ... AdminBasedeDatos [WSL: Ubuntu]

etl_pipeline.py etl_pyspark.py X

practica_etl > notebooks > etl_pyspark.py > ...

```
1 """
2 Práctica: Análisis de Datos – ETL con PySpark
3 Actividad 5 (variante PySpark) + Actividad 7 con MLlib
4 """
5
6 from pyspark.sql import SparkSession
7 from pyspark.sql import functions as F
8 from pyspark.sql.types import DoubleType, IntegerType
9 from pyspark.ml.feature import VectorAssembler, StringIndexer
10 from pyspark.ml.regression import LinearRegression, GBRegressor
11 from pyspark.ml import Pipeline
12 from pyspark.ml.evaluation import RegressionEvaluator
13 import warnings
14 from pathlib import Path
15 warnings.filterwarnings("ignore")
16
17 # Rutas relativas al script
18 BASE_DIR = Path(__file__).resolve().parent.parent
19 DATA_DIR = BASE_DIR / "data"
20 DATA_DIR.mkdir(parents=True, exist_ok=True)
21
22 #
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

bash - practica_etl

```
(venv) miguel@miguel:~/abd/AdminBasedeDatos/practica_etl$ python notebooks/etl_pipeline.py && python notebooks/etl_pyspa
rk.py
|Monitor    |Electrónica|4094.45|237.5 |972431.88 |798739.42 |
|Arroz      |Electrónica|6197.88|484.0 |2999773.92|3031709.43|
|Playera    |Alimentos  |3510.14|113.0 |396645.82 |612463.01 |
|Cojin      |Deportes   |623.36 |129.0 |80413.44  |201774.82 |
|Yogurt     |Deportes   |351.2  |290.0 |101848.0  |160097.85 |
+-----+-----+-----+-----+-----+
only showing top 10 rows

✓ PySpark ETL + MLlib completado
(venv) miguel@miguel:~/abd/AdminBasedeDatos/practica_etl$
```

WSL: Ubuntu main 0 0 0 0 0 0 MonkeyFlip (Hace 11 minutos) Lin. 5, col. 1 Espacios: 4 UTF-8 LF {} Python venv (3.12.3) Prettier